

Kubernetes Q & A

Kubernetes architecture overview

Kubernetes follows a master-worker [architecture](#). The control plane (formerly master node) manages cluster operations while worker nodes run containerized applications. Pods, the smallest deployable unit in Kubernetes, run containers and are assigned to nodes.

Kubernetes ensures the desired state by continuously monitoring and adjusting workloads as required.

Core components of Kubernetes:

Control Plane manages the cluster and makes scheduling decisions.

Worker Nodes execute and run containerized applications.

Explain the Kubernetes Control Plane and its Components.

The Control Plane is responsible for managing the Kubernetes cluster. It receives user requests, schedules workloads, stores cluster information, and maintains the desired state. Components:

[API Server](#): The entry point of Kubernetes. It accepts REST API requests and coordinates communication among cluster components.

[etcd](#): A distributed key-value database that stores the cluster configuration and state. It is the source of truth for Kubernetes.

[Scheduler](#): Selects the most suitable Worker Node for newly created Pods based on available resources and scheduling policies.

Controller Manager: Runs controllers that continuously compare the current state with the desired state and take corrective actions when required.

Explain the Worker Node and its Components.

A Worker Node is responsible for running containerized applications. Each Worker Node contains components required to execute and manage Pods. Components:

[Kubelet](#): An agent that communicates with the API Server and ensures Pods are running correctly.

[kube-proxy](#): Manages networking and enables communication between Services and Pods.

Container Runtime: Responsible for pulling container images and running containers. Common runtimes include containerd and CRI-O.

What is a Pod in Kubernetes?

A Pod is the smallest deployable unit in Kubernetes. It represents one or more containers running in a shared environment. Containers inside a Pod share networking and storage resources.

Here's a simple Pod definition YAML file:

```
apiVersion: v1
kind: Pod
metadata:
  name: my-pod
spec:
  containers:
```

- name: nginx-container

image: nginx:1.21

ports:

- containerPort: 80

Explain the Pod Lifecycle.

A Pod passes through different phases during its [lifecycle](#).

Pending: Pod has been accepted but is waiting to start.

Running: At least one container is running.

Succeeded: All containers completed successfully.

Failed: One or more containers terminated with an error.

Unknown: Kubernetes cannot determine the Pod status.

What is the purpose of kubectl?

Kubectl is the primary CLI tool for managing Kubernetes resources and interacting with the cluster. Here are a few common kubectl commands you should be familiar with:

kubectl get pods # list all Pods

kubectl get services # list all Services

kubectl logs <pod-name> # view logs of a Pod

kubectl exec -it <pod-name> -- /bin/sh # open a shell inside a Pod

What is a ReplicaSet?

A [ReplicaSet](#) ensures that a specified number of Pod replicas are always running. If a Pod fails or is deleted, it automatically creates a new Pod to maintain the desired state. Key Points: Maintains desired number of Pods, Provides self-healing, Usually managed by a Deployment, Not commonly created directly

Explain Deployment Strategies.

[Deployment strategies](#) define how a new application version replaces the old one while minimizing downtime and risk.

Rolling Update: Gradually replaces old Pods with new Pods while keeping the application available. Best for: Most production deployments.

Recreate: Stops all existing Pods before starting new Pods. Best for: Applications that cannot run multiple versions simultaneously.

Blue-Green Deployment: Runs two identical environments: Blue (current version), Green (new version). Traffic is switched to the Green environment after validation.

Canary Deployment: Releases the new version to a small percentage of users before gradually rolling it out to everyone. Benefits Lower deployment risk, Early issue detection, Gradual rollout.

What is a Rollback?

[Rollback](#) restores the previous stable version of an application if a deployment fails or introduces issues.

Kubernetes performs the rollback by reverting to the previous ReplicaSet. Benefits Quick recovery, Reduced downtime, Safer application updates.

What is Scaling in Kubernetes?

[Scaling](#) is the process of increasing or decreasing application resources to handle changes in workload. Kubernetes supports scaling by adjusting the number of Pods or the cluster's infrastructure. Types of Scaling Horizontal Scaling, Vertical Scaling, Cluster Scaling.

What is the Horizontal Pod Autoscaler (HPA)?

The [Horizontal Pod Autoscaler \(HPA\)](#) automatically increases or decreases the number of Pod replicas based on metrics such as CPU utilization, memory usage, or custom metrics. Example Traffic Increases Traffic Increases -> CPU Usage-> HPA Adds More Pods

What is the Cluster Autoscaler?

The [Cluster Autoscaler](#) automatically adds or removes Worker Nodes based on resource demand. It increases the cluster size when Pods cannot be scheduled and removes unused nodes when demand decreases. Pods Pending -> No Worker Node Available -> Cluster Autoscaler Adds Node

Difference Between HPA, VPA, and Cluster Autoscaler

HPA: Pods, Number of replicas, CPU, Memory, Custom Metrics, Stateless applications.
VPA: Pod Resources, CPU & Memory, Resource recommendations, Resource optimization.
Cluster Autoscaler: Worker Nodes, Number of Nodes, Unschedulable Pods, Cluster capacity management.

What are Labels and Selectors?

Labels: Labels are key-value pairs attached to Kubernetes objects such as Pods, Services, and Deployments. They help identify, organize, and categorize resources. Example:

labels:

app: nginx
environment: production

Selectors: Selectors are used to identify Kubernetes objects based on their labels. Services and ReplicaSets use selectors to locate the Pods they should manage.

selector:

app: nginx

What are the Types of Kubernetes Services?

Kubernetes provides different [Service types](#) based on how an application should be accessed.

ClusterIP: Exposes the application only within the cluster. It is the default Service type. Use Case: Communication between internal microservices.

NodePort: Exposes the application on a fixed port of every Worker Node, allowing external access using the node's IP address and port. Use Case: Development and testing environments.

LoadBalancer: Creates an external load balancer provided by the cloud platform and assigns a public IP address to the Service. Use Case: Production applications running on cloud providers.

ExternalName: Maps a Kubernetes Service to an external DNS name without creating a proxy. Use Case: Accessing external databases or third-party services.

Headless Service: A Headless Service does not assign a Cluster IP. Instead, it returns the IP addresses of individual Pods, allowing direct communication. Use Case: Stateful applications such as databases.

Explain Helm Charts and Helm Repositories.

Helm Chart: A Helm Chart is a collection of YAML templates, configuration files, and metadata that defines a Kubernetes application. It allows applications to be packaged and deployed consistently.

Helm Repository: A Helm Repository is a collection of Helm Charts that can be shared and downloaded. It functions similarly to a software package repository.

What is ImagePullBackOff, and how do you troubleshoot it?

ImagePullBackOff occurs when Kubernetes cannot pull the required container image from the container registry. Common Causes Incorrect image name or tag, Private registry authentication failure, Image does not exist, Network connectivity issues, Registry unavailable. Troubleshooting:

```
kubectl describe pod <pod-name>
kubectl get events
```

What is OOMKilled?

OOMKilled (Out of Memory Killed) occurs when a container exceeds its configured memory limit. The Linux kernel terminates the container to protect the node from running out of memory. Common Causes Low memory limits, Memory leaks, High application memory usage.

Why does a Pod remain in the Pending state?

A Pod remains in the Pending state when Kubernetes cannot schedule or start it successfully. Common Causes Insufficient CPU or memory, No suitable Worker Node, Unbound Persistent Volume Claim (PVC), Taints without matching tolerations, Unsatisfied node affinity rules. Troubleshooting:

```
kubectl describe pod <pod-name>
kubectl get nodes
kubectl get pvc
kubectl get events
```

What is a Deployment in Kubernetes?

A Deployment in Kubernetes is a higher-level abstraction that manages the lifecycle of Pods. It ensures that the desired number of replicas are up and running and provides features like rolling updates, rollbacks, and self-healing.

Here’s how a simple Deployment definition YAML file looks like:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deployment
spec:
  replicas: 3
  selector:
    matchLabels:
```

```
  app: nginx
  template:
  metadata:
    labels:
  app: nginx
  spec:
    containers:
      - name: nginx
  image: nginx:1.21
  ports:
    - containerPort: 80
```

What is a Kubernetes Service, and why is it needed?

A Service in Kubernetes exposes a group of Pods and allows communication between and to them. Since Pods are ephemeral, their IPs can change, meaning the application talking to the Pods must also change the IP address. Services, therefore, provide a stable network endpoint with a fixed IP address.

A simple Service YAML definition:

```
apiVersion: v1
kind: Service
metadata:
  name: my-service
spec:
  selector:
  app: my-app
  ports:
    - protocol: TCP
      port: 80
      targetPort: 80
  type: ClusterIP
```

What is the role of ConfigMaps and secrets in Kubernetes?

ConfigMaps stores non-sensitive configuration data, while secrets stores sensitive data like API keys and passwords.

Using secrets allows you to avoid putting secret information into your application code. In contrast, ConfigMaps will enable you to make your applications more configurable, as these values can be easily edited, and you don't need to persist them in your application code.

What are Namespaces in Kubernetes?

A Namespace is a virtual cluster within a Kubernetes cluster. It helps to organize workloads in multi-tenant environments by isolating resources within a cluster.

What are Labels and Selectors in Kubernetes?

Labels are key/value pairs attached to objects (e.g. Pods). They help to organize Kubernetes objects. Selectors filter resources based on Labels.

What are Persistent Volumes (PVs) and Persistent Volume Claims (PVCs)?

Persistent Volumes (PVs) provide storage that persists beyond Pod lifecycles. The PV is a storage piece in the cluster that has been provisioned by a cluster administrator or dynamically provisioned using Storage Classes. **A Persistent Volume Claim (PVC)** is a request for storage by a user.

What is Kubernetes networking, and how does it work?

Kubernetes networking allows for communication between Pods, Services, and external clients. It follows a flat networking structure, meaning that, by default, all Pods can communicate with each other. Key networking concepts in Kubernetes include:

Pod-to-pod communication: Each Pod gets a unique IP assigned and can communicate within the cluster.

Service-to-pod communication: Services provide a stable network endpoint for a group of Pods, as Pods are ephemeral. Each pod gets a new IP assigned every time it is created.

Ingress controllers: Manage external HTTP/HTTPS traffic.

Network policies: Define rules to restrict or allow communication between Pods.

What is Role-Based Access Control (RBAC) in Kubernetes?

RBAC is a security mechanism that restricts users and services based on their permissions. It consists of: Roles and ClusterRoles: Define the actions allowed on resources.

RoleBindings and ClusterRoleBindings: Assign roles to users or service accounts.

The following YAML shows an example role that only allows read-only access to Pods:

```
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: pod-reader
rules:
  - apiGroups: ["" ]
    resources: ["pods"]
    verbs: ["get", "watch", "list"]
```

How does Kubernetes autoscaling work?

Kubernetes provides three types of autoscaling to optimize resource usage:

Horizontal Pod Autoscaler (HPA): Adjusts the number of Pods based on CPU usage, memory usage, or custom metrics.

Vertical Pod Autoscaler (VPA): Adjusts the CPU and memory requests for individual Pods.

Cluster Autoscaler: Adjusts the number of worker nodes in the cluster based on resource needs.

You can create an HPA using kubectl:

```
kubectl autoscale deployment nginx --cpu-percent=50 --min=1 --max=10
```

How do you debug Kubernetes Pods?

When Pods fail, Kubernetes provides multiple ways to debug it:

Use `kubectl logs` to check container logs for errors.

Use `kubectl describe pod` to inspect events and recent state changes.

Use `kubectl exec` to open an interactive shell and investigate from inside the container.

Use `kubectl get pods --field-selector=status.phase=Failed` to list all failing Pods.

What is the difference between liveness and readiness probes?

Both are health checks Kubernetes runs against containers, but they serve different purposes:

Liveness probe: Detects if a container is alive but stuck (e.g., deadlocked). If the probe fails, Kubernetes restarts the container. Use it for crash and deadlock recovery.

Readiness probe: Detects if a container is ready to receive traffic. If it fails, Kubernetes removes the Pod from the Service's endpoint list without restarting it. Use it to prevent traffic being routed to a Pod that hasn't fully started yet.

```
apiVersion: v1
```

```
kind: Pod
```

```
metadata:
```

```
  name: my-app
```

```
spec:
```

```
  containers:
```

```
    - name: my-app
```

```
  image: my-app:v2
```

```
  livenessProbe:
```

```
    httpGet:
```

```
      path: /healthz
```

```
      port: 8080
```

```
        initialDelaySeconds: 10
```

```
        periodSeconds: 5
```

```
  readinessProbe:
```

```
    httpGet:
```

```
      path: /ready
```

```
      port: 8080
```

```
        initialDelaySeconds: 5
```

```
        periodSeconds: 3
```

How do you perform rolling updates and rollbacks in Kubernetes?

Kubernetes Deployments support rolling updates to avoid downtime. You can perform a rolling update by editing a Deployment or explicitly setting its image to a new version using:

```
kubectl set image deployment/my-deployment nginx=nginx:1.21
```

You can then check the Deployment status:

```
kubectl rollout status deployment my-deployment
```

[Powered By](#)

If you want to roll back to the previous version, you can run:

```
kubectl rollout undo deployment my-deployment
```

What is an Ingress in Kubernetes, and how does it work?

An Ingress is an API object that manages external HTTP/HTTPS access to Services inside a Kubernetes cluster. It allows routing requests based on hostname and paths, acting as a reverse proxy for multiple applications.

Example Ingress YAML definition:

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: my-ingress
spec:
  rules:
  - host: my-app.example.com
    http:
      paths:
      - path: /
        pathType: Prefix
    backend:
      service:
        name: my-service
        port:
          number: 80
```

How does Kubernetes handle resource limits and requests?

Kubernetes allows you to set resource requests and limits for Pods to keep resource allocation balanced and avoid overusing cluster resources.

Requests are the minimum amount of CPU and memory a Pod needs. They are permanently assigned to a Pod.

Limits are the maximum a Pod can use. They are not assigned to the Pod, but if it requires more resources, it can grow until the limit is reached.

Example YAML Pod definition that sets resource requests and limits:

```
apiVersion: v1
kind: Pod
metadata:
  name: resource-limited-pod
spec:
  containers:
  - name: my-container
    image: nginx
    resources:
      requests:
        memory: "256Mi"
```

```
cpu: "250m"
```

```
limits:
```

```
memory: "512Mi"
```

```
cpu: "500m"
```

What are StatefulSets, and how do they differ from Deployments?

A **StatefulSet** is used for stateful applications that require stable network identities, persistent storage, and ordered deployment. Unlike Deployments, StatefulSets ensure that:

Pods have stable and unique network identities, where Pods are named like pod-0, pod-1 etc.

Pods are created, updated, and deleted in order.

Each Pod retains persistent storage across restarts. Persistent storage is defined as part of the StatefulSet YAML definition.

How can you secure a Kubernetes cluster?

Follow the 4C security model to secure a Kubernetes cluster:

Cloud provider security: Use IAM roles and firewall rules.

Cluster security: Enable RBAC, audit logs, and API server security.

Container security: Scan images and use non-root users.

Code security: Implement secrets management and use network policies.

What are Taints and Tolerations in Kubernetes?

Taints and Tolerations control where Pods run:

Taints: Mark nodes to reject Pods.

Tolerations: Allow Pods to ignore certain Taints.

Here's an example for tainting a node only to accept specific workloads:

```
kubectl taint nodes node1 key1=value1:NoSchedule
```

This means no Pod can be scheduled on node1 until it has the required Toleration.

A Toleration is added in the PodSpec section like the following:

```
apiVersion: v1
```

```
kind: Pod
```

```
metadata:
```

```
  name: nginx
```

```
  labels:
```

```
env: test
```

```
spec:
```

```
  containers:
```

```
    - name: nginx
```

```
image: nginx
```

```
imagePullPolicy: IfNotPresent
```

```
  tolerations:
```

```
    - key: "key1"
```

```
operator: "Equal"
```

value: "value1"

effect: "NoSchedule"

The Pod would be allowed to be scheduled on node1.

What are Kubernetes sidecar containers, and how are they used?

A sidecar container runs alongside the main application container inside the same Pod. It is commonly used for:

Logging and monitoring (e.g. collecting logs with Fluentd).

Security proxies (e.g. running Istio's Envoy proxy for service mesh).

Configuration management (e.g. syncing application settings).

What is a DaemonSet, and when would you use it?

A DaemonSet ensures that one copy of a specific Pod runs on every node in the cluster (or a subset of nodes). When a new node joins, the DaemonSet schedules a Pod on it automatically. When a node is removed, that Pod is garbage collected.

Common DaemonSet use cases:

Log collection agents — Fluentd or Logstash running on each node to collect container logs centrally.

Monitoring agents — Prometheus Node Exporter or Datadog Agent, which need visibility into every node's metrics.

Network plugins — CNI plugins like Calico or Flannel that require a presence on every node to manage networking.

How do you safely upgrade a Kubernetes cluster?

Upgrading a Kubernetes cluster is a multi-step process that requires minimal downtime and maintains cluster stability. Administrators should follow a structured approach to prevent issues during the upgrade:

Drain and back up etcd before the upgrade to ensure that you can restore it in case the upgrade fails.

Upgrade the control plane node.

Upgrade kubelet and kubectl on control plane nodes.

Upgrade worker nodes one by one. Before upgrading, each worker node must be drained to prevent workload disruption.

Upgrade cluster add-ons (e.g., Ingress controllers, monitoring tools, ...).

How do you monitor a Kubernetes cluster?

Kubernetes administrators must monitor CPU, memory, disk, networking, and application health. The following tools are recommended for these tasks:

Prometheus + Grafana: Collect and visualize cluster metrics. Create real-time alerts to get notified in case there are issues.

Loki + Fluentd: Collect and analyze logs.

Kubernetes dashboard: UI-based cluster monitoring.

Jaeger/OpenTelemetry: Distributed tracing.

How do you secure a Kubernetes cluster?

Security is a key aspect, and every administrator should follow best practices to secure a Kubernetes cluster:

Enable RBAC: Restrict user access using Roles and RoleBindings.

Pod security admission: Use admission controllers to enforce the Pod security standards.

Enforce NetworkPolicies: Restrict Pod communication, as by default, all Pods can communicate with each other.

Rotate API tokens and certificates regularly.

Use secrets management: Use tools like Vault, AWS Secrets Manager, etc.

How do you implement high availability in Kubernetes?

High availability is essential to avoid downtime of applications running in your Kubernetes cluster. You can ensure high availability by:

Using multiple control plane nodes. Multiple API servers prevent downtime if one fails.

Enabling the cluster autoscaler. This automatically adds/removes nodes based on demand.

Debugging a slow Kubernetes application

“Your team reports that an application running in Kubernetes has become slow, and users are experiencing high response times. How do you solve this problem?”

You can approach the problem using the following steps:

1. Check Pod resource utilization. Increase resources in case they are at their limits.

```
kubectl top pods --sort-by=cpu
```

```
kubectl top pods --sort-by=memory
```

2. Describe the slow Pod to get more information. Look for resource throttling, restart counts, or liveness probe failures.

```
kubectl describe pod <pod-name>
```

3. Check container logs for errors. Look for timeouts, errors, or connection failures.

```
kubectl logs <pod-name>
```

4. Test network latency, as they can slow down applications.

```
kubectl exec -it <pod-name> -- ping my-database
```

```
kubectl exec -it <pod-name> -- curl http://my-service
```

5. Verify Kubernetes node health and check for resource exhaustion on nodes.

```
kubectl get nodes
```

```
kubectl describe node <node-name>
```

Kubernetes Deployment fails after an upgrade

“You deployed a new version of your application, but it fails to start, causing downtime for your users. How can you fix the problem?”

Approach to tackle the problem:

1. Roll back to the previous working version.

```
kubectl rollout undo deployment my-app
```

2. Check the Deployment history and identify what has changed in the new version.

```
kubectl rollout history deployment my-app
```

3. Check the new Pod’s logs for errors.

kubectl logs -l app=my-app

4. Check readiness and liveness probes.

5. Verify image-pulling issues. Sometimes, the new image is wrong or unavailable.

A pod is in CrashLoopBackOff. How do you debug it?

kubectl logs <pod>

kubectl describe pod <pod> → check events, probes.

Verify environment variables, config maps.

Check for OOMKilled in kubectl get pod -o wide.

Run interactively with kubectl exec -it.

How does Kubernetes handle node failure in AWS (EKS)?

Node stops sending heartbeats → marked NotReady.

Pods become Unknown → rescheduled by controllers.

EKS Node Group or Auto Scaling Group (ASG) may automatically replace failed nodes.

Use PodDisruptionBudgets (PDBs) to maintain availability.

How do you implement zero-downtime deployments?

Set:

strategy:

rollingUpdate:

maxUnavailable: 0

maxSurge: 1

Use readinessProbes and preStop hooks.

Use canary or blue/green deployment patterns (e.g., via Argo Rollouts).

How do you secure secrets in Kubernetes on AWS?

Enable encryption at rest with AWS KMS via EKS configuration.

Use TLS for in-transit encryption.

Apply fine-grained access control via RBAC.

Use External Secrets Operator to sync from AWS Secrets Manager or Parameter Store.

Difference between LoadBalancer and Ingress in AWS?

LoadBalancer: Allocates an AWS ELB (Classic/NLB/ALB) per service.

Ingress: Uses a single ALB (via AWS Load Balancer Controller) to route traffic to multiple services.

What happens if the API server is down in EKS?

No new resources can be created or modified.

Existing workloads continue running.

Kubelets on nodes keep managing running pods independently.

How do you configure a new cluster in kubectl for EKS?

```
aws eks update-kubeconfig --name <cluster-name> --region <region>bash
```

For production Kubernetes workloads, I follow a structured approach focused on availability, security, scalability, and monitoring.

First, I provision the cluster using Infrastructure as Code tools such as Terraform and ensure worker nodes are distributed across multiple availability zones for high availability.

Next, I configure namespaces, RBAC policies, network policies, and secrets management to enforce security and isolate workloads properly.

For application deployment, I use Deployments, Services, ConfigMaps, and Secrets. I define resource requests and limits to prevent resource contention and ensure cluster stability.

I implement liveness and readiness probes so Kubernetes can automatically restart unhealthy containers and route traffic only to healthy pods.

For scalability, I configure Horizontal Pod Autoscaler and Cluster Autoscaler to automatically handle changes in traffic and resource demand.

Monitoring and logging are critical in production, so I implement solutions such as Prometheus, Grafana, ELK stack, or CloudWatch to track application health, performance metrics, and logs. Alerts are configured for proactive issue detection.

For deployments, I use rolling updates, blue-green deployments, or canary releases to minimize risk and support fast rollback if issues occur.

Finally, I perform regular cluster upgrades, security patching, backup and disaster recovery testing, and capacity reviews to ensure the platform remains secure, reliable, and production-ready.

I integrate Jenkins or GitLab CI/CD with Kubernetes by automating the complete build, test, security scan, and deployment lifecycle.

Whenever a developer commits code to Git, the pipeline is triggered automatically. The pipeline first performs code quality checks, unit tests, and security scans. After successful validation, it builds a Docker image and pushes it to a container registry such as ECR, Docker Hub, or GitLab Container Registry.

In the deployment stage, the pipeline updates the Kubernetes manifests or Helm chart with the new image version and deploys it to the target Kubernetes cluster using kubectl or Helm.

For rolling deployments, Kubernetes gradually replaces old pods with new pods while maintaining service availability. Health checks and readiness probes ensure traffic is routed only to healthy pods.

For canary deployments, I use tools such as Argo Rollouts, Istio, or NGINX Ingress. Initially, only a small percentage of traffic, such as 10%, is routed to the new version. Monitoring tools like Prometheus and Grafana are used to track error rates, latency, and application health. If the metrics remain healthy, traffic is gradually increased to 50% and then 100%. If issues are detected, the deployment is automatically rolled back to the previous stable version.

This approach minimizes deployment risk, reduces downtime, and enables safer releases in production environments.

For building and managing Docker containers in production, I follow several best practices focused on security, performance, scalability, and maintainability.

First, I use lightweight base images such as Alpine or official runtime images to reduce image size and improve startup time. I also use multi-stage builds to separate the build environment from the runtime environment, which keeps production images smaller and more secure.

I avoid hardcoding configuration values and secrets inside images. Instead, I use environment variables, Kubernetes Secrets, AWS Secrets Manager, or Vault for secure configuration management.

From a security perspective, I run containers as non-root users, regularly update base images, and perform vulnerability scanning using tools such as Trivy, Snyk, or ECR image scanning.

I implement proper image versioning and tagging strategies instead of relying on the 'latest' tag so deployments are traceable and rollback is easier.

For observability, container logs are written to stdout and stderr and collected by centralized logging solutions such as CloudWatch, ELK, or Splunk.

During deployment, I use health checks, resource limits, and autoscaling policies to ensure application stability. Docker images are built and deployed through CI/CD pipelines, and Kubernetes or ECS manages scaling and self-healing.

These practices help improve reliability, security, and operational efficiency in production environments.